TÀI LIỆU ĐẶC TẢ THIẾT KẾ HỆ THỐNG

HỆ THỐNG HỎI ĐÁP VÀ TRA CỨU VĂN BẢN QUY PHẠM PHÁP LUẬT VIỆT NAM

Thông tin Quản lý Thiết kế	Chi tiết Kỹ thuật
Tên Hệ thống	Hệ thống Hỏi đáp Văn bản Quy phạm Pháp luật
Mã Tài liệu	SDD-LEGAL-RAG-001
Tài liệu Liên kết	SRS-LEGAL-RAG-001 Phiên bản 2.0
Phiên bản	4.0
Ngày Ban hành	25/10/2026
Tiêu chuẩn Áp dụng	IEEE 1016-2009

CHƯƠNG 1. KIÊN TRÚC HỆ THÔNG TÔNG THÊ

1.1. Mục tiêu Thiết kế và Nguyên tắc Kiến trúc

Tài liệu này mô tả thiết kế kỹ thuật chi tiết theo chuẩn IEEE 1016-2009 để hiện thực hóa các yêu cầu trong SRS-LEGAL-RAG-001. Hệ thống được thiết kế theo mô hình kiến trúc phân tán kết hợp giữa máy trạm cục bộ và hạ tầng đám mây miễn phí nhằm tối ưu tài nguyên phần cứng:

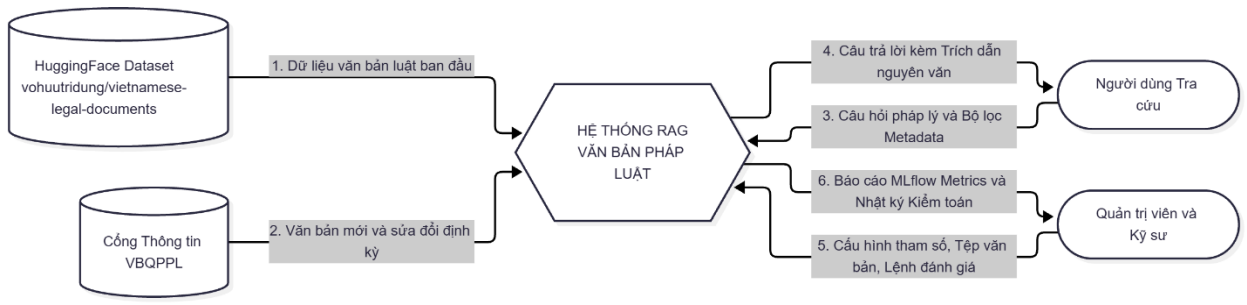
1.2. Bảng Đặc tả Công nghệ Thực thi

Hệ thống chốt cố định một bộ công nghệ duy nhất để triển khai xuyên suốt:

Tầng Kiến trúc	Công nghệ Sử dụng	Môi trường Chạy	Vai trò Kỹ thuật Cụ thể
Nguồn Dữ liệu Nền	HuggingFace datasets	Google Colab / Máy cục bộ	Tải luông dữ liệu từ kho vohuutridung/vietnamese-legal-documents.
Bộ Cào Định kỳ	Playwright, BeautifulSoup4	GitHub Actions Cron	Quét văn bản mới ban hành hàng tuần và kiểm tra mã băm SHA-256.
Bộ Phân tích Văn bản	IBM Docling, PyMuPDF, Regex	Google Colab / Máy cục bộ	Bóc tách văn bản theo phân cấp Chương, Mục, Điều, Khoản và giữ bảng Markdown.
Bộ Phân tích Truy vấn	Python Regex AST, Aho-Corasick	Máy trạm Cục bộ	Bộ Phân tích Truy vấn
Mô hình Embedding	BAAI/bge-m3 Fine-Tuned	Google Colab GPU / ONNX INT8 CPU	Mô hình Embedding
Mô hình Reranker	BAAI/bge-reranker-v2-m3 Fine-Tuned	ONNX Runtime INT8 trên CPU	Mô hình Reranker
Huấn luyện Tinh chỉnh	PyTorch, FlagEmbedding, Optimum ONNX	Google Colab GPU T4	Huấn luyện Tinh chỉnh
Cơ sở Dữ liệu Vector	Qdrant Cloud	Cloud Free-Tier	Lưu trữ vector kép, đánh chỉ mục HNSW và Inverted Index, hỗ trợ lọc Metadata.
Cơ sở Dữ liệu Quan hệ	Neon PostgreSQL 16	Cloud Free-Tier	Lưu danh mục văn bản gốc, trạng thái hiệu lực, lịch sử cào và nhật ký kiểm toán.

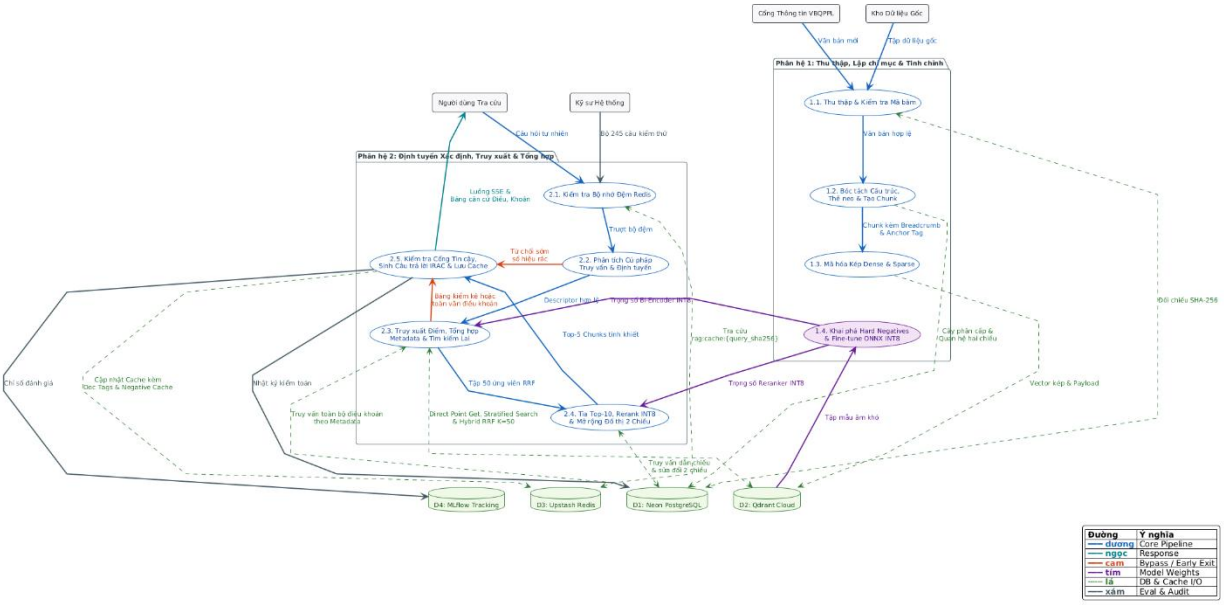
Tầng Kiến trúc	Công nghệ Sử dụng	Môi trường Chạy	Vai trò Kỹ thuật Cụ thể
Bộ đệm và Khóa	Upstash Redis 7	Cloud Free-Tier	Lưu bộ đệm kết quả truy vấn và tạo khóa phân tán chống chạy trùng bộ cào.
Mô hình Ngôn ngữ Lớn	Qwen2.5-7B-Instruct Q4_K_M	Máy trạm Cục bộ	Sinh câu trả lời pháp lý có trích dẫn dựa trên ngữ cảnh được cung cấp.
Engine Phục vụ LLM	Ollama Server	Máy trạm Cục bộ	Phục vụ mô hình ngôn ngữ lớn qua cổng 11434, chia sẻ tải giữa GPU 4GB và RAM.
Điều phối và Backend	FastAPI, LangGraph, langchain-core	Máy trạm Cục bộ	Điều phối đồ thị trạng thái RAG, kiểm tra ngưỡng tin cậy và phát luồng SSE.
Giao diện Người dùng	Chainlit	Máy trạm Cục bộ	Hiển thị khung hội thoại thời gian thực, bộ lọc pháp lý và thẻ đối chiếu trích dẫn.
Đánh giá Chất lượng	MLflow	Máy trạm Cục bộ	Ghi nhận kết quả đo lường Recall@25, Hit Rate@5, MRR@5 và Faithfulness.

1.3. Khung nhìn Ngữ cảnh và Luồng Dữ liệu Mức 0



1.4. Khung nhìn Phân rã Luồng Dữ liệu Mức 1

Phụ lục A.1

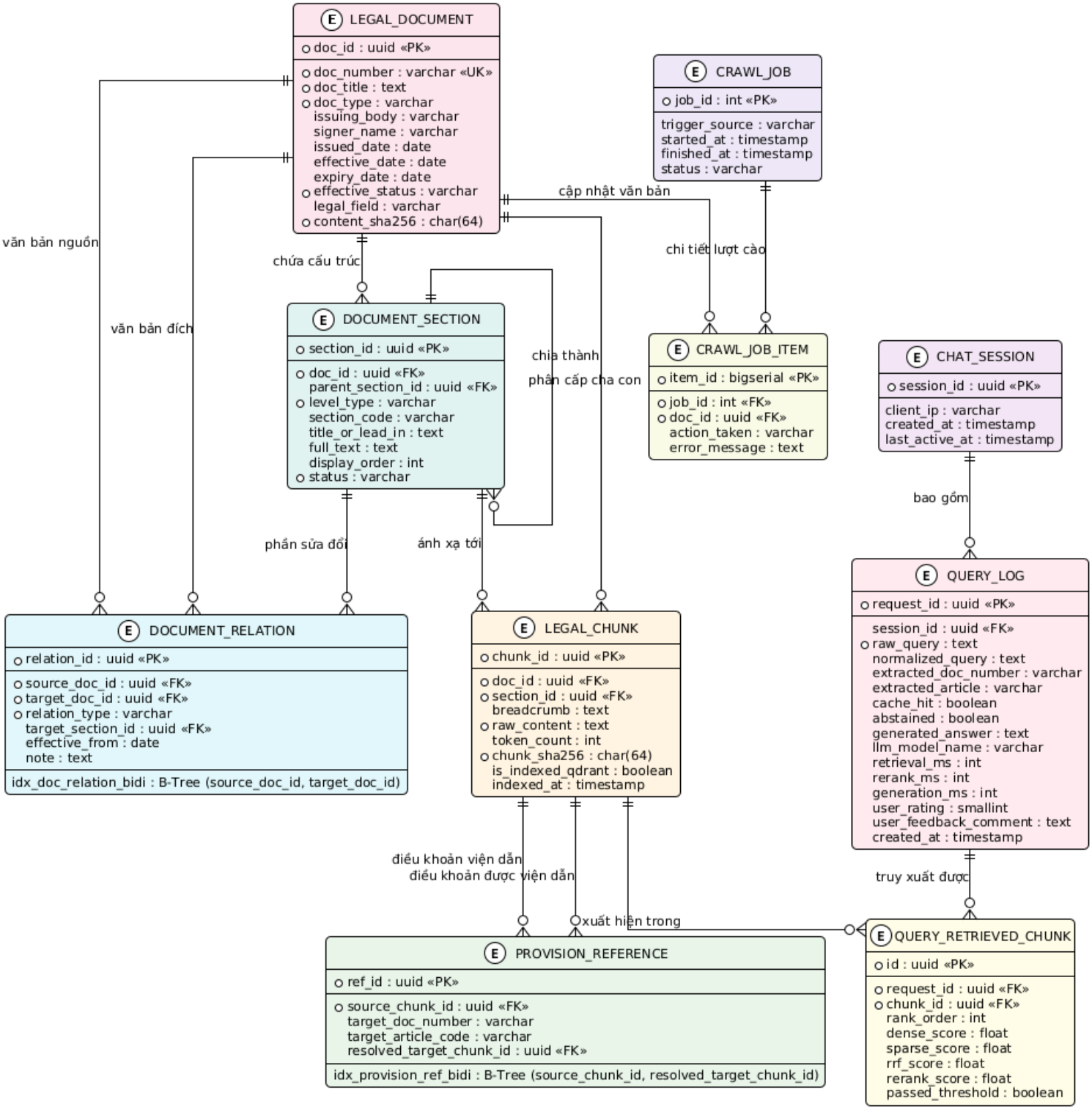


CHƯƠNG 2. THIẾT KẾ DỮ LIỆU VÀ LƯU TRỮ

2.1. Chiến lược Phân tầng Lưu trữ

- Tầng Quan hệ trên Neon PostgreSQL lưu danh mục toàn bộ văn bản quy phạm pháp luật, giữ vai trò đối chiếu mã băm SHA-256 và quản lý trạng thái hiệu lực.
- Tầng Vector trên Qdrant Cloud chỉ lưu trữ các chunk thuộc nhóm Bộ luật, Luật, Nghị định, Thông tư có trạng thái hiệu lực là `active` hoặc `amended`, kết hợp bật lượng tử hóa `INT8 Scalar Quantization` để giữ dung lượng chỉ mục dưới 4GB Disk và 1GB RAM.

2.2. Entity Relation Diagram



2.3. Thiết kế Collection và Payload trên Qdrant Cloud

Collection `legal_chunks_v1` trên Qdrant được thiết kế với cấu hình Named Vectors kết hợp Payload Indexing:from qdrant_client import QdrantClient, models

Tên Vector	Kiểu Biểu diễn	Số chiều	Độ đo Khoảng cách	Thuật toán Chỉ mục & Tham số	Tối ưu Bộ nhớ
dense_vector	Dense Float Vector	1024	Cosine Similarity	HNSW ($M = 16$, $ef_construct = 200$)	Scalar Quantization INT8
sparse_vector	Sparse Lexical Vector	Biến thiên	Inner Product	Inverted Index	Không áp dụng

Tên Trường Payload	Kiểu Dữ liệu	Chỉ mục Lọc (Payload Index)	Mục đích Sử dụng
doc_id	UUID	UUID Index	Khóa ngoại liên kết bảng legal_documents trong PostgreSQL.
doc_number	String	Keyword Index	Lọc chính xác theo số hiệu văn bản (Ví dụ: 100/2019/NĐ-CP).
doc_type	String	Keyword Index	Lọc theo loại văn bản (Bộ luật, Luật, Nghị định, Thông tư).
issued_year	Integer	Integer Index	Lọc hoặc phân vùng văn bản theo năm ban hành.
effective_status	String	Keyword Index	Lọc trạng thái hiệu lực (active, partially_amended, expired).
article_code	String	Keyword Index	Lọc chính xác theo số Điều (Ví dụ: Điều 6).
clause_code	String	Keyword Index	Lọc theo số Khoản (Ví dụ: Khoản 2 hoặc ALL).
breadcrumb	String	Không đánh chỉ mục	Chuỗi phân cấp nguồn gốc dùng hiển thị ngữ cảnh nhanh.
chunk_sha256	String	Keyword Index	Kiểm tra tính bất biến và chống trùng lặp khi nạp dữ liệu.
legal_field	String	Keyword Index	Lọc theo lĩnh vực pháp lý phục vụ truy vấn tổng hợp và phân vùng tìm kiếm.
anchor_tag	String	Không đánh chỉ mục	Chuỗi định danh ngắn `[doc_number:article_code]` ghép vào đầu đoạn trích khi chạy Reranker.

2.4. Thiết kế Không gian Khóa trên Redis

Tiền tố Khóa (Key Pattern)	Cấu trúc Định danh	Kiểu Dữ liệu Redis	Thời gian Sống (TTL)	Mục đích Kỹ thuật
rag:cache:query:{query_hash}	Mã băm SHA-256 của câu hỏi đã chuẩn hóa kèm bộ lọc	String (JSON)	86.400 giây (24h)	Lưu kết quả trả lời và trích dẫn để phản hồi ngay khi trúng bộ đệm.
rag:lock:{job_name}	Tên tiến trình nền (Ví dụ: periodic_crawler)	String	3.600 giây (1h)	Khóa phân tán đảm bảo chỉ có duy nhất 1 tiến trình cào chạy tại một thời điểm.
rag:tag:doc:{doc_number}	Số hiệu văn bản quy phạm pháp luật	Set	86.400 giây	Lưu tập hợp các khóa `query_sha256` có trích dẫn văn bản để thu hồi bộ đệm khi văn bản hết hiệu lực.
rag:tag:negative_cache	Nhãn quản lý bộ đệm từ chối toàn cục	Set	600 giây	Quản lý danh sách các khóa truy vấn bị từ chối do thiếu căn cứ hoặc sai số hiệu.

CHƯƠNG 3. THIẾT KẾ CHI TIẾT CÁC PHÂN HỆ

3.1. Phân hệ Bóc tách Cấu trúc Phân cấp và Sinh Chunk (`parser_service.py`)

3.1.1. Cấu trúc Dữ liệu Đầu ra của mỗi Chunk

```
class LegalChunkRecord(BaseModel):
    chunk_id: str = Field(..., description="Mã UUID5 sinh từ doc_number + article_id + clause_id")
    doc_id: str
    doc_number: str
    doc_title: str
    doc_type: str
    issuing_body: Optional[str] = None
    issued_year: int
    effective_status: str = "active"
    chapter_id: Optional[str] = None
```

section_id: Optional[str] = None
article_id: str
clause_id: str = "ALL"
referenced_provisions: List[str] = []
raw_content: str
anchor_tag: str

3.1.2. Thuật toán State Machine Cắt Chunk và Bảo toàn Câu dẫn

1. Đọc văn bản đầu vào từng dòng sau khi đã chuẩn hóa bảng mã Unicode NFC và chuyển đổi bảng biểu sang Markdown.
2. Duy trì bộ nhớ trạng thái hiện hành gồm: `current_chapter`, `current_section`, `current_article_id`, `current_article_title`, `article_lead_in` và danh sách `clauses`.
3. Khi gặp dòng khớp biểu thức `^Điều\s+(\d+[a-zđ]?)\.\s*(.*)$`:
   - Đóng gói `Điều` trước đó thành các đối tượng `LegalChunkRecord`.
   - Khởi tạo trạng thái cho `Điều` mới; các dòng văn bản xuất hiện trước `Khoản 1` được tích lũy vào biến `article_lead_in`.
4. Quy tắc đóng gói một `Điều`:
   - Đếm tổng số token của toàn bộ `Điều` bằng bộ đếm token của `BAAI/bge-m3`.
   - Nếu tổng số token không vượt quá 1024: Tạo một bản ghi duy nhất với `clause_id = "ALL"`.
   - Nếu tổng số token vượt quá 1024: Tách mỗi `Khoản` thành một bản ghi riêng với `clause_id = "Khoản X"`, trong đó phần nội dung của mỗi bản ghi được ghép theo công thức:

`Breadcrumb + "\n" + Tiêu đề Điều + "\n" + article_lead_in + "\n" + Nội dung Khoản X`.

3.2. Phân hệ Mã hóa Kép và Đồng bộ Vector (`embedding_service.py`)

3.2.1. Mã hóa Kép và Đồng bộ Vector

- Sử dụng mô hình `BAAI/bge-m3` với tham số `return_dense=True, return_sparse=True, max_length=1024`.
- Đối với vector thưa: Chuyển đổi từ điển trọng số từ vựng `{token_id: weight}` của `bge-m3` sang định dạng `models.SparseVector(indices=list_of_token_ids, values=list_of_weights)` tương thích trực tiếp với Qdrant.

- Khóa chính `chunk_id` trong Qdrant được sinh bằng `uuid.uuid5(uuid.NAMESPACE_OID, f"{doc_number}_{article_id}_{clause_id}")` để đảm bảo tính bất biến khi thực thi lệnh `upsert`.

3.2.2. Quy trình Tinh chỉnh Đối chiếu và Lượng tử hóa ONNX INT8

1. Khai phá mẫu âm khó: Với mỗi cặp câu hỏi huấn luyện và điều khoản đích, thực hiện truy vấn Top-25 trên Qdrant. Các đoạn trích nằm trong Top-25 nhưng không thuộc tập điều khoản đích được thu thập làm mẫu âm khó theo tỷ lệ 7 mẫu âm cho mỗi mẫu dương.
2. Tinh chỉnh BAAI/bge-m3: Huấn luyện đối chiếu bằng hàm mục tiêu InfoNCE kết hợp đồng thời điểm Dense và trọng số Sparse với hệ số nhiệt độ $\tau = 0.02$: $L_{InfoNCE} = -\log [\exp(s(q, p+) / \tau) / (\exp(s(q, p+) / \tau) + \Sigma \exp(s(q, p-_j) / \tau))]$
3. Tinh chỉnh BAAI/bge-reranker-v2-m3: Huấn luyện bằng hàm mất mát Binary Cross-Entropy trên tập bộ ba câu hỏi, điều khoản đích và mẫu âm khó cùng văn bản để tách biệt biên độ điểm số.
4. Lượng tử hóa và cắt ngưỡng 512 tokens: Xuất cả hai mô hình sang định dạng ONNX INT8. Khi chấm điểm qua Int8RerankerAnchor512Node, ghép anchor_tag vào đầu văn bản, cắt độ dài tối đa 512 tokens và áp dụng đệm lô động theo độ dài thực tế lớn nhất trong lô.

3.3. Phân hệ Phân tích Truy vấn, Truy xuất Phân nhánh và Mở rộng Đồ thị

3.3.1. Phân tích Cú pháp Truy vấn và Bóc tách Bộ lọc

Nút StructuredQueryDescriptorNode phân tích câu hỏi đầu vào trước khi truy cập cơ sở dữ liệu:

- Chuẩn hóa từ vựng: Quét từ điển Aho-Corasick để chuyển đổi từ lóng, phương ngữ và từ viết tắt sang thuật ngữ pháp quy; tự động chuyển sang tìm kiếm Dense thuần túy nếu phát hiện câu hỏi thuộc nhóm ngôn ngữ tượng hình.
- Trích xuất và kiểm tra số hiệu: Quét biểu thức chính quy để trích xuất danh sách doc_filters và article_filter. Nếu năm ban hành trong số hiệu văn bản lớn hơn max_issued_year hiện hành, kích hoạt nhánh AbstainFastExitNode và ghi bộ đệm từ chối TTL 600 giây.
- Xác định cờ is_aggregation: Chỉ bật is_aggregation = True khi từ khóa thống kê đi kèm danh từ đơn vị cấu trúc văn bản hoặc điều khoản. Nếu từ khóa đếm đi kèm danh từ nghiệp vụ nội hàm như khung giờ, cấp độ hoặc trường hợp, giữ is_aggregation = False và lược bỏ từ khóa đếm khỏi chuỗi tìm kiếm.

- Luôn đính kèm điều kiện lọc mặc định loại trừ các văn bản có effective_status = "expired".

3.3.2. Thực thi Truy xuất Phân nhánh theo Đặc tả

Dựa trên kết quả của StructuredQueryDescriptorNode, hệ thống định tuyến xác định vào 1 trong 4 nhánh:

1. Nhánh Đích danh Tuyệt đối (len(doc_filters) == 1 và article_filter != None và is_aggregation == False):
    - Truy xuất trực tiếp điểm dữ liệu trên Qdrant qua DirectPointGetNode.
    - Bắt buộc thực thi BiDirectionalGraphExpandNode để kiểm tra và nạp các điều khoản sửa đổi, bổ sung văn bản gốc trước khi chuyển tới cổng tin cậy.
2. Nhánh Tổng hợp Diện rộng (is_aggregation == True):
    - Thực thi FetchAllMetadataProvisionsNode truy vấn danh sách điều khoản thỏa mãn điều kiện lọc từ PostgreSQL.
    - Nếu tổng số điều khoản lớn hơn 15: Thực thi GenerateHierarchicalInventoryTableNode để tạo bảng kiểm kê phân cấp dưới 600 tokens kèm toàn văn Top-5 điều khoản tiêu biểu, đồng thời giữ toàn bộ danh sách định danh điều khoản trong siêu dữ liệu trích dẫn.
    - Nếu tổng số điều khoản từ 15 trở xuống: Thực thi AttachAllVerbatimClausesNode đính kèm toàn văn các điều khoản thu được.
3. Nhánh So sánh Đối chiếu Đa văn bản (len(doc_filters) >= 2):
    - Thực thi StratifiedMinQuotaSearchNode sử dụng điều kiện lọc should kết hợp định mức sàn tối thiểu 5 đoạn trích cho mỗi văn bản để bảo toàn ngữ cảnh của tất cả văn bản cần so sánh.
4. Nhánh Tìm kiếm Lai Ngữ nghĩa (ScopedHybridRrfPrefetch50Node):
    - Truy vấn đồng thời nhánh Dense và nhánh Sparse trên Qdrant với giới hạn thu thập K = 50 ứng viên mỗi nhánh, hợp nhất thứ hạng bằng công thức Reciprocal Rank Fusion với hằng số k = 60.

3.3.3. Cắt tỉa Top-10, Tái xếp hạng INT8 và Mở rộng Đồ thị Hai chiều

Đối với hai nhánh tìm kiếm vector, danh sách 50 ứng viên RRF được xử lý qua chuỗi 4 bước:

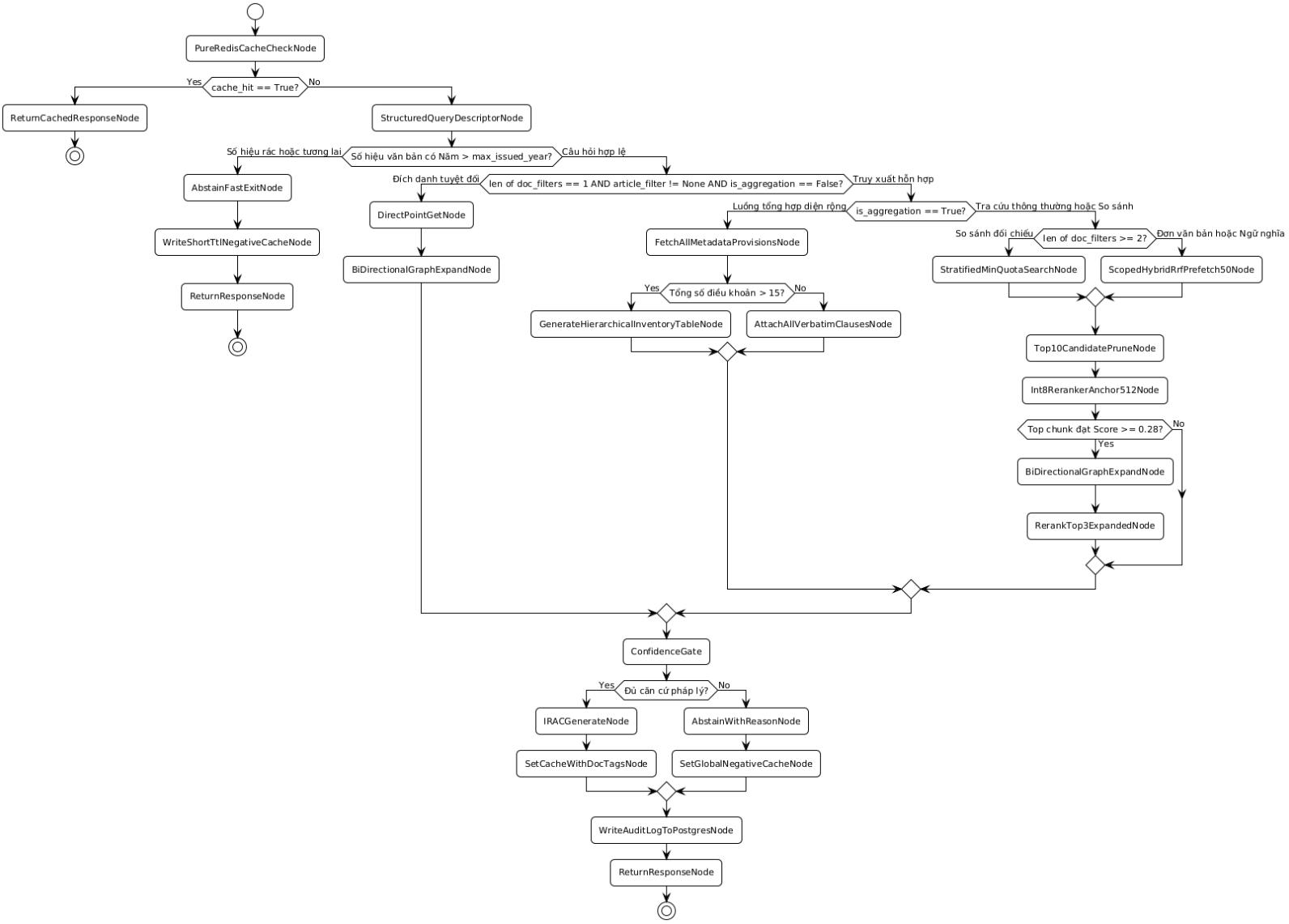
1. Cắt tỉa ứng viên: Top10CandidatePruneNode giữ lại 10 ứng viên có điểm RRF cao nhất.
2. Chấm điểm Cross-Encoder lần một: Int8RerankerAnchor512Node chấm điểm 10 ứng viên ở độ dài tối đa 512 tokens kèm thẻ neo [doc:art] và chuẩn hóa điểm qua hàm Sigmoid về đoạn [0, 1].
3. Mở rộng đồ thị hai chiều có điều kiện: Nếu ứng viên cao điểm nhất đạt Score >= 0.28, thực thi BiDirectionalGraphExpandNode truy vấn bảng PROVISION_REFERENCE và DOCUMENT_RELATION trên PostgreSQL để lấy tối đa 3 điều khoản có quan hệ dẫn chiếu xuôi hoặc sửa đổi ngược.
4. Tái chấm điểm mở rộng: Thực thi RerankTop3ExpandedNode chấm điểm các điều khoản liên đới vừa nạp, hợp nhất với tập ứng viên hiện có và lọc lấy Top-5 đoạn trích tinh khiết với tổng độ dài ngữ cảnh không vượt quá 2.500 tokens.

3.4. Phân hệ Điều phối Đồ thị Trạng thái (`orchestrator_service.py`)

3.4.1. Cấu trúc Trạng thái Đồ thị LangGraph

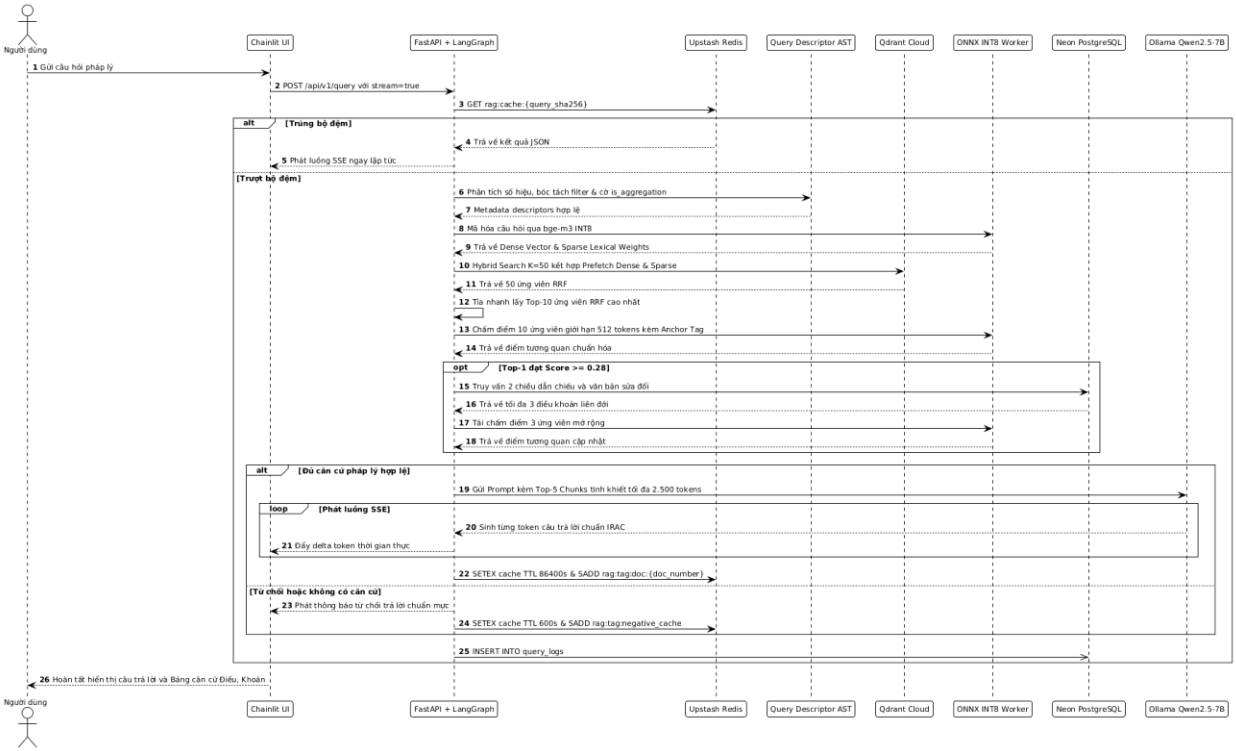
```
class LegalRAGState(TypedDict):
    request_id: str
    query: str
    query_sha256: st
    cache_hit: bool
    doc_filters: List[str]
    article_filter: Optional[str]
    is_aggregation: bool
    invalid_future_doc: bool
    candidate_chunks: List[Dict[str, Any]]
    expanded_chunks: List[Dict[str, Any]]
    top_chunks: List[Dict[str, Any]]
    inventory_table: Optional[str]
    abstained: bool
    answer: str
    citations: List[Dict[str, Any]]
    latency_ms: Dict[str, int]
```

3.4.2. LangGraph Orchestration Activity Diagram



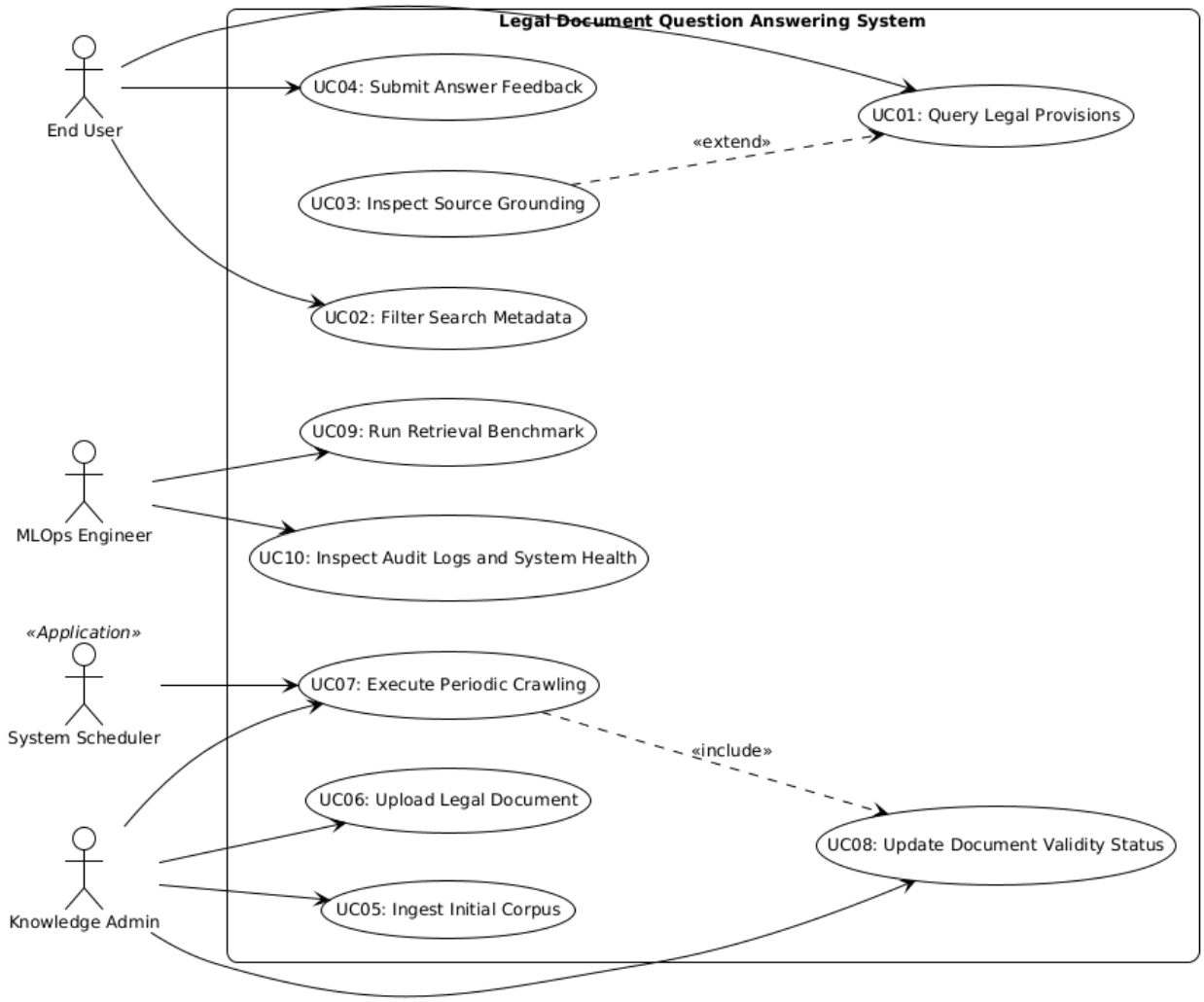
3.4.3. Biểu đồ Tuần tự Luồng Thực thi Truy vấn

Phụ lục A.2



CHƯƠNG 4. THIẾT KẾ GIAO DIỆN TÍCH HỢP VÀ API

4.1. Biểu đồ Use Case Tổng thể



4.2. Đặc tả Các Điểm cuối RESTful API

Phương thức	Đường dẫn Endpoint	Mô tả Chức năng	M
POST	`/api/v1/query`	Thực thi truy vấn hỏi đáp pháp luật, trả về JSON hoặc luồng SSE.	2
POST	`/api/v1/ingest`	Tải lên tệp văn bản `.docx/.pdf`, bóc tách cấu	2

Phương thức	Đường dẫn Endpoint	Mô tả Chức năng	M
		trúc và nạp lên Qdrant và PostgreSQL.	
PATCH	`/api/v1/documents/{doc_id}/status`	Cập nhật trạng thái hiệu lực của văn bản trên PostgreSQL và Qdrant Payload.	2
POST	`/api/v1/feedback`	Ghi nhận đánh giá của người dùng cho một `request_id` cụ thể.	2
GET	`/api/v1/health`	Kiểm tra kết nối tới Ollama Local, Qdrant Cloud, Neon PostgreSQL và Redis.	2

4.3. Giao thức Truyền tải Luồng Sự kiện SSE (`POST /api/v1/query` với `stream=true`)

Khi tham số `stream=true`, máy chủ trả về tiêu đề `Content-Type: text/event-stream` với 3 loại sự kiện tuần tự:

1. Sự kiện `metadata` (Gửi ngay sau khi hoàn tất bước Rerank):
   ```
   event: metadata
   data: {"request_id": "req_9021a8f4", "abstained": false,
   "citations": [...]}
   ```
2. Sự kiện `token` (Lặp lại cho mỗi token sinh ra từ Ollama):
   ```
   event: token
   data: {"delta": "Phạt tiền từ 400.000 đồng"}
   ```
3. Sự kiện `done` (Kết thúc luồng và trả thông số độ trễ):
   ```
   event: done
   data: {"metrics": {"retrieval_latency_ms": 110,
   "rerank_latency_ms": 280, "generation_latency_ms": 3200,
   "total_latency_ms": 3590}}
   ```

4.4. Phương pháp luận Kiến tạo Tập dữ liệu Kiểm thử Chuẩn

4.4.1. Quy trình Tổng hợp Truy vấn Ngược có Kiểm chứng Trực tiếp

Tập dữ liệu kiểm thử gồm 245 kịch bản được xây dựng qua quy trình 3 bước tự động:

1. Lấy mẫu phân tầng từ chỉ mục Qdrant: Sử dụng giao thức Scroll API trích xuất các điểm dữ liệu từ tập 199.042 đoạn trích trên Qdrant Cloud, phân nhóm theo số hiệu văn bản, loại văn bản, năm ban hành và lĩnh vực để chọn tập điều khoản hạt nhân.
2. Tổng hợp truy vấn ngược theo 14 hình thái: Từ nội dung điều khoản hạt nhân, mô hình ngôn ngữ lớn sinh câu hỏi ngược theo 14 khuôn mẫu đặc tả gồm câu hỏi ẩn số hiệu văn bản, hồ sơ tình huống dài, câu hỏi xâu chuỗi đa văn bản, câu hỏi đa ngôn ngữ, câu hỏi không dấu chứa phương ngữ và câu hỏi thống kê tập hợp.
3. Kiểm định chân lý nền trực tiếp: Toàn bộ 205 truy vấn chuẩn được thực thi lệnh kiểm định trực tiếp trên Qdrant Cloud để xác nhận 100% điều khoản mục tiêu tồn tại thực tế trong chỉ mục; 40 truy vấn kiểm thử từ chối được xác nhận không tồn tại điều khoản tương ứng.

4.4.2. Ma trận Phân bổ Tập Dữ liệu Kiểm thử 245 Kịch bản

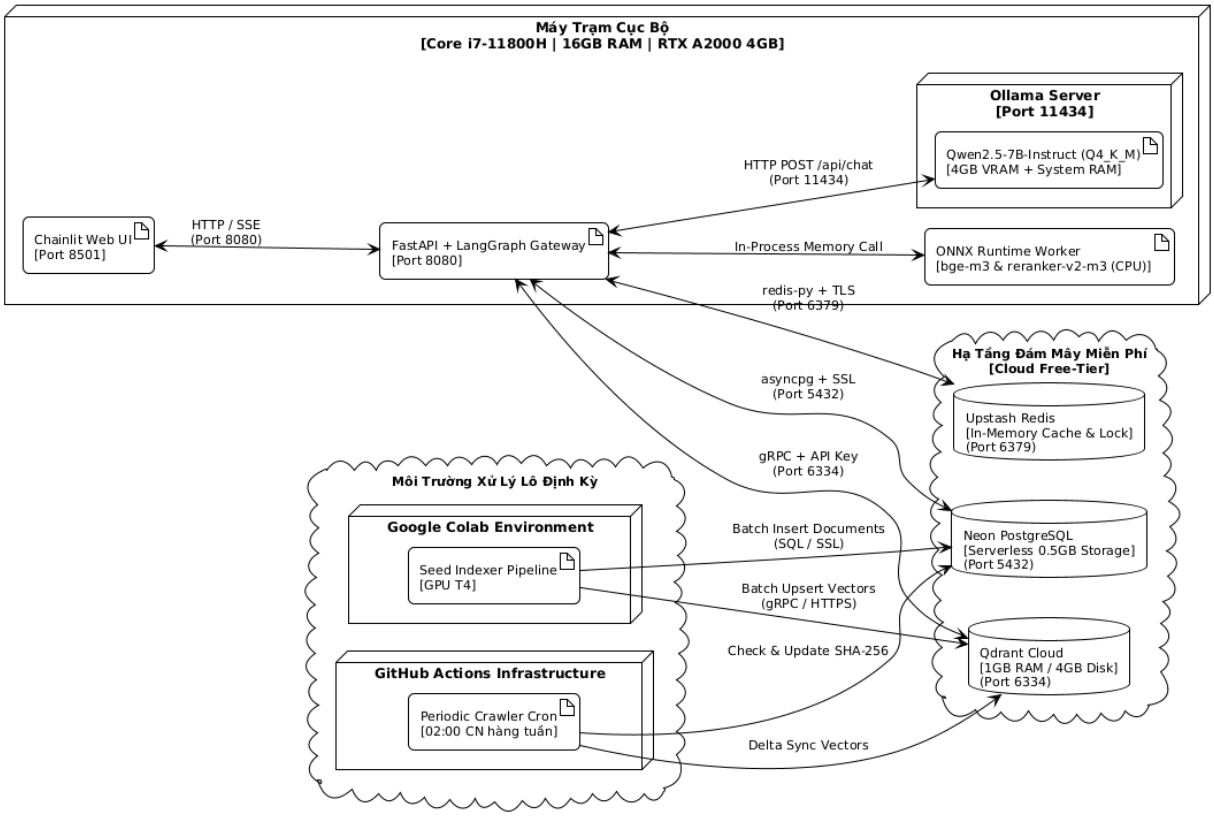
Nhóm Đánh giá	Danh mục Kiểm thử	Số lượng	Độ dài từ	Số Đích / Câu	Mục tiêu Kiểm định Kỹ thuật
Tra cứu Ngữ nghĩa	semantic	58	15 – 100	1	Khớp ngữ nghĩa tự nhiên khi ẩn số hiệu văn bản.
Kiểm soát Ảo giác	negative_abstention	40	10 – 60	0	Kiểm định thoát nhanh số hiệu tương lai và cổng từ chối ngoài phạm vi.
Truy vấn Tập hợp	aggregation	30	12 – 95	2 – 188	Kiểm định truy vấn siêu dữ liệu PostgreSQL và nén bảng kiểm kê phân cấp.

Nhóm Đánh giá	Danh mục Kiểm thử	Số lượng	Độ dài từ	Số Đích / Câu	Mục tiêu Kiểm định Kỹ thuật
Suy luận Sâu & Tình huống	multistep_reasoning	30	25 – 65	1 – 2	Kiểm định mở rộng đồ thị 2 chiều liên kết nhiều thông tư, nghị định.
Suy luận Sâu & Tình huống	lawyer_case_study	20	100 – 147	1 – 2	Kiểm định tỉa ứng viên, chấm điểm thẻ neo 512 tokens và suy luận IRAC.
Suy luận Sâu & Tình huống	theoretical_jurisprudence	5	30 – 55	1 – 2	Kiểm định lý luận pháp lý về hiệu lực hồi tố, xung đột thẩm quyền và án lệ.
Độ bền & Biên	cross_lingual	15	15 – 45	1 – 2	Kiểm định tìm kiếm đa ngôn ngữ trên 6 thứ tiếng.
Độ bền & Biên	summarization_synthesis	15	20 – 45	1 – 2	Kiểm định tìm kiếm phân tầng bảo toàn định mức tối thiểu trên từng văn bản.
Độ bền & Biên	ambiguous_keyword	12	1 – 95	1	Kiểm định khả năng xử lý truy vấn từ khóa ngắn hoặc mơ hồ.
Độ bền & Biên	temporal_conflict	10	25 – 50	1 – 2	Kiểm định phát hiện văn bản sửa đổi ngược và ưu tiên quy định mới.

Nhóm Đánh giá	Danh mục Kiểm thử	Số lượng	Độ dài từ	Số Đích / Câu	Mục tiêu Kiểm định Kỹ thuật
Độ bền & Biên	noisy_colloquial	10	18 – 35	1	Kiểm định chuẩn hóa từ lóng pháp lý và truy vấn tiếng Việt không dấu.

CHƯƠNG 5. THIẾT KẾ TRIỂN KHAI VÀ QUY TRÌNH THỰC HIỆN

5.1. Sơ đồ Triển khai Vật lý và Kết nối Mạng



5.2. Cấu trúc Thư mục Mã nguồn Dự án

```
legal-rag-system/

├─── .env.example
├─── requirements.txt
├─── app/
│    ├─── main.py
│    ├─── config.py
│    ├─── schemas/
│    │    ├─── api_models.py
│    │    ├─── chunk_models.py        # Cập nhật thêm anchor_tag
│    │    └─── descriptor_models.py    # Định nghĩa cấu trúc dữ liệu đầu ra của Query Descriptor AST
│    ├─── services/
│    │    ├─── parser_service.py
│    │    ├─── descriptor_service.py   # Phân tích số hiệu, kiểm tra năm tương lai và cờ is_aggregation
│    │    ├─── embedding_service.py    # Mã hóa và chấm điểm Reranker INT8 512 tokens kèm thẻ neo
│    │    ├─── retriever_service.py    # Thực thi Direct Get, Stratified Quota Search và Hybrid RRF K=50
│    │    ├─── graph_expand_service.py # Mở rộng đồ thị 2 chiều dẫn chiếu/sửa đổi và tổng hợp bảng kiểm kê
│    │    ├─── llm_service.py         # Giao tiếp Ollama cổng 11434 theo khuôn mẫu suy luận IRAC
│    │    ├─── cache_service.py       # Quản lý bộ đệm theo thẻ văn bản và bộ đệm từ chối TTL 600s
│    │    └─── db_service.py
│    └─── workflow/
│         └─── langgraph_rag.py       # Đồ thị điều phối đơn luồng xác định theo thiết kế mới
├─── scripts/
│    ├─── init_databases.py          # Tạo bảng PostgreSQL, chỉ mục đồ thị 2 chiều và Collection Qdrant
│    ├─── colab_seed_indexer.ipynb
│    ├─── mine_hard_negatives.py     # Khai phá mẫu âm khó từ Qdrant phục vụ tinh chỉnh
```

```
 |    ├──── finetune_and_export_onnx.py # Huấn luyện bge-m3, reranker và xuất mô hình ONNX INT8
 |    ├──── synthesize_benchmark.py     # Sinh tập dữ liệu kiểm thử 245 câu thuộc 14 danh mục
 |    ├──── periodic_crawler.py         # Cào văn bản mới, cập nhật hiệu lực và thu hồi Redis Doc Tag
 |    └──── evaluate_mlflow.py          # Đo kiểm 245 kịch bản kiểm thử chuẩn và ghi log vào MLflow
 └──── ui/
      └──── chainlit_app.py
```

5.3. Đặc tả Biến Môi trường Hệ thống (`.env`)

1. Cấu hình Cơ sở dữ liệu Vector Qdrant Cloud

```
QDRANT_HOST="<qdrant-cluster-hostname>"

QDRANT_PORT=6333

QDRANT_API_KEY="<qdrant-secret-api-key>"

QDRANT_COLLECTION="legal_chunks_v1"
```

2. Cấu hình Cơ sở dữ liệu Quan hệ Neon PostgreSQL

```
DATABASE_DSN="postgresql+asyncpg://<user>:<password>@<neon-hostname>/legal_db?sslmode=require"
```

3. Cấu hình Bộ đệm Upstash Redis

```
REDIS_DSN="rediss://default:<upstash-password>@<upstash-hostname>:6379"

CACHE_TTL_SECONDS=86400
NEGATIVE_CACHE_TTL_SECONDS=600
```

4. Cấu hình Máy chủ Suy luận Cục bộ Ollama

```
OLLAMA_HOST="127.0.0.1:11434"
```

OLLAMA_MODEL_NAME="qwen2.5:7b-instruct-q4_K_M"

LLM_TEMPERATURE=0.1

LLM_TOP_P=0.90

LLM_MAX_TOKENS=1024

5. Cấu hình Mô hình Truy xuất và Ngưỡng Kiểm soát

EMBEDDING_MODEL_PATH="models/bge-m3-finetuned-int8"

RERANKER_MODEL_PATH="models/bge-reranker-v2-m3-finetuned-int8"

PREFETCH_LIMIT_K=50

STRATIFIED_MIN_QUOTA_PER_DOC=5

TOP_K_CANDIDATE_PRUNE=10

RERANK_MAX_TOKENS=512

GRAPH_EXPAND_TRIGGER_SCORE=0.28

GRAPH_EXPAND_MAX_RELATIONS=3

AGGREGATION_VERBATIM_LIMIT=15

INVENTORY_TABLE_MAX_TOKENS=600

MAX_CONTEXT_TOKENS=2500

MAX_VALID_ISSUED_YEAR=2026

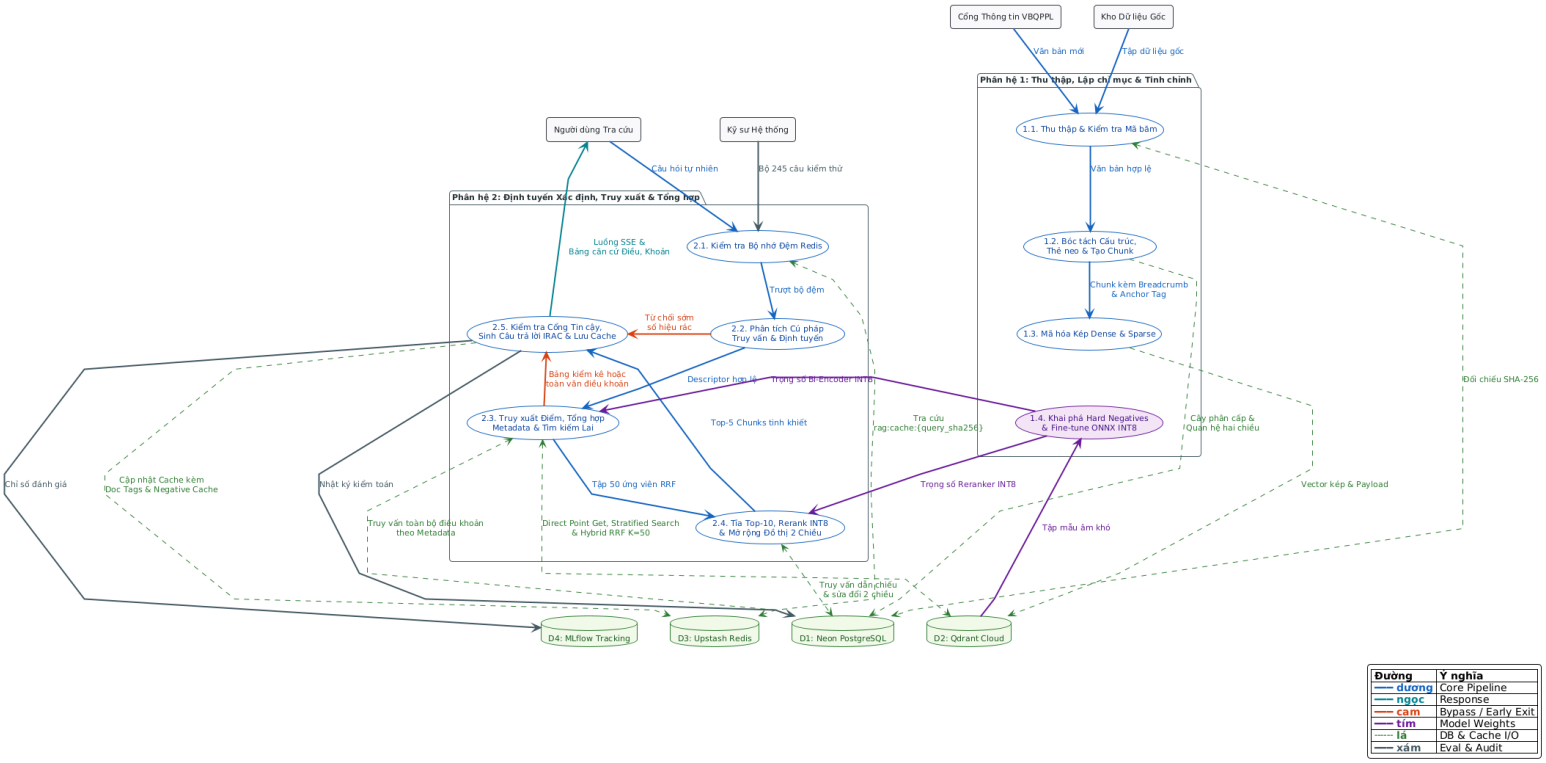
5.4. Quy trình Triển khai Kỹ thuật Từng bước

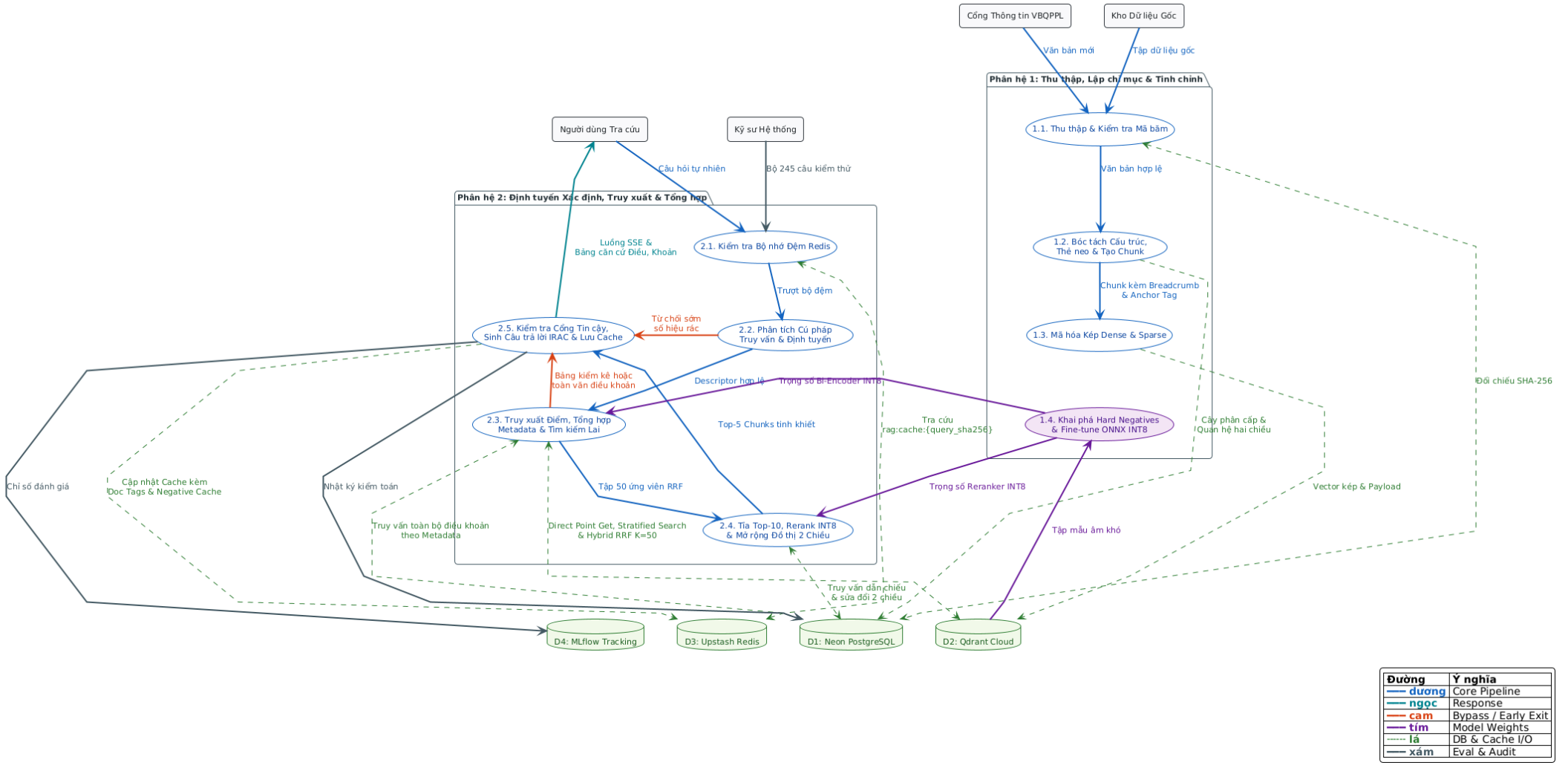
- **Bước 1 — Khởi tạo Đám mây và Lược đồ:** Đăng ký cụm miễn phí trên Qdrant Cloud, Neon PostgreSQL và Upstash Redis; điền thông số vào tệp `.env` và chạy `python scripts/init_databases.py` để tạo bảng PostgreSQL cùng Collection `legal_chunks_v1` có bật nén `INT8 Scalar Quantization`.
- **Bước 2 — Nạp Kho Dữ liệu Nền trên Google Colab:** Mở tệp `scripts/colab_seed_indexer.ipynb` trên Google Colab (chọn GPU T4), đọc luồng dữ liệu từ `vohuutridung/vietnamese-legal-documents`, lọc các văn bản quy phạm còn hiệu lực, chạy `parser_service.py` để cắt theo `Điều/Khoản` kèm `Breadcrumb`, mã hóa bằng `BAAI/bge-m3` và đẩy trực tiếp lên Qdrant Cloud cùng Neon PostgreSQL.
- **Bước 3 — Cài đặt Mô hình Cục bộ:** Cài đặt Ollama trên máy trạm, thực thi lệnh `ollama pull qwen2.5:7b-instruct-q4_K_M` và tải trọng số ONNX của `bge-m3` và `bge-reranker-v2-m3` về thư mục cục bộ.
- **Bước 4 — Kiểm định Tầng Truy xuất với MLflow:** Chạy `python scripts/evaluate_mlflow.py` trên bộ 50 câu hỏi kiểm thử chuẩn để xác nhận `Recall@25 >= 92%` và `Hit Rate@5 >= 85%`.
- **Bước 5 — Khởi chạy Backend và Giao diện:** Chạy máy chủ FastAPI bằng lệnh `uvicorn app.main:app --port 8080` và khởi động giao diện người dùng bằng lệnh `chainlit run ui/chainlit_app.py --port 8501`.
- **Bước 6 — Nghiệm thu Toàn trình:** Thực thi kiểm thử 5 bộ kịch bản từ `SUITE-A` đến `SUITE-E` theo đúng tiêu chí của tài liệu SRS.

Phụ lục

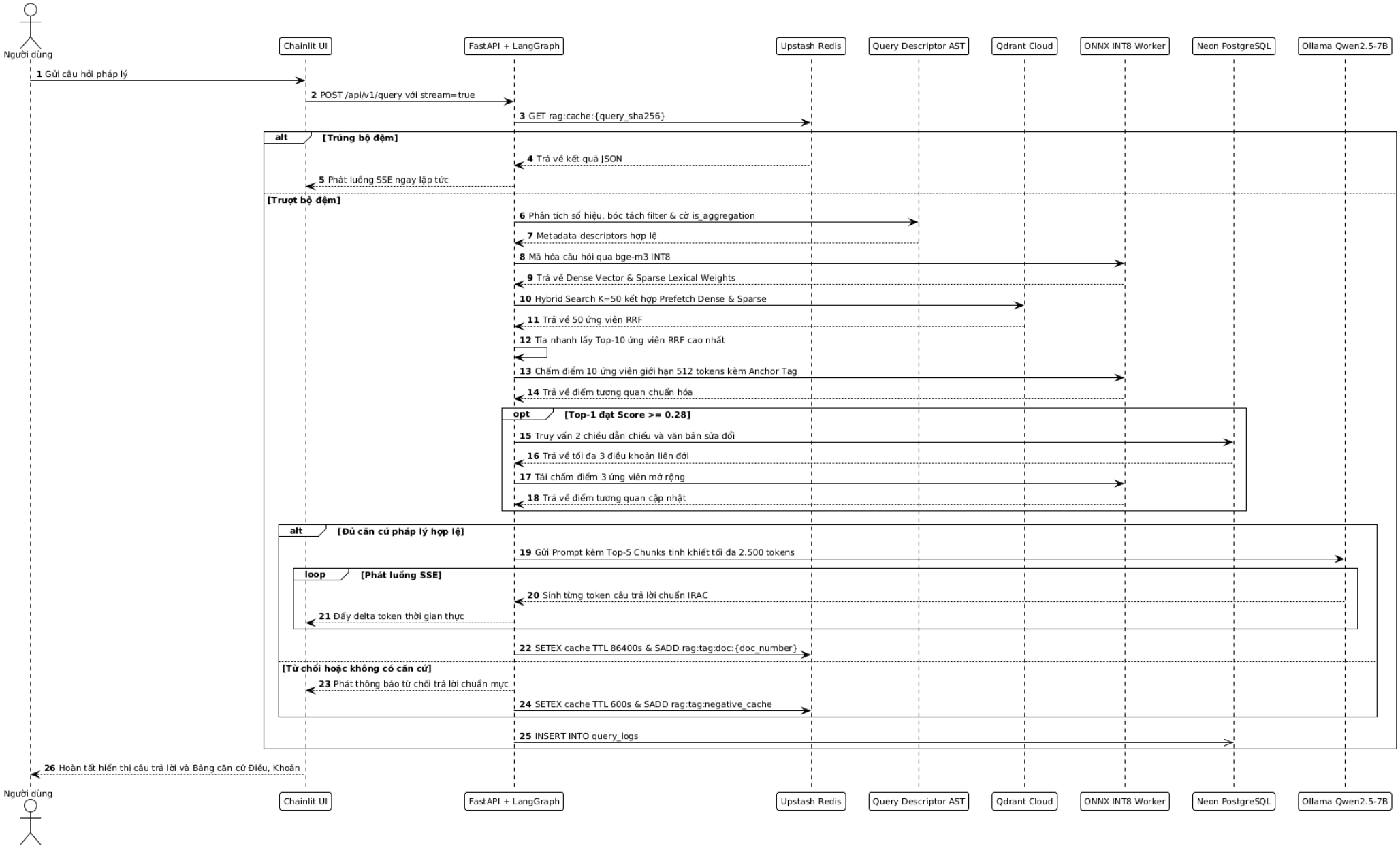
A. Các biểu đồ

1. Biểu đồ Khung nhìn Phân rã Luồng Dữ liệu Mức 1





Đường	Ý nghĩa
dương	Core Pipeline
ngọc	Response
cam	Bypass / Early Exit
tím	Model Weights
lá	DB & Cache I/O
xám	Eval & Audit



2. Biểu đồ luồng hoạt động